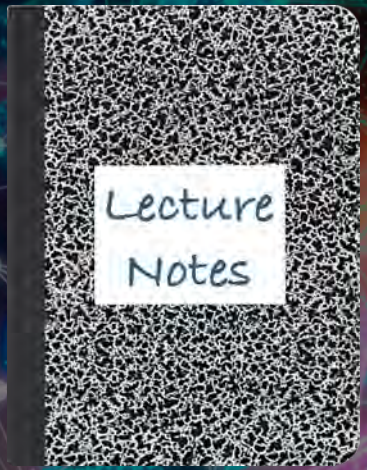


CS 419 – Computer Security

Week 3

Integrity and Public Key Cryptography

Paul Krzyzanowski

© 2026 Paul Krzyzanowski. No part of this content may be reproduced or reposted in whole or in part in any manner without the permission of the copyright owner.

What We Will Cover

- Detecting that data changed with a **trustworthy** check value

  Hashes

- Adding a secret key so an outsider cannot replace the check value

  Message Authentication Codes

- Splitting the key so anyone can verify and only one party can sign

  Public Key Cryptography & Digital Signatures

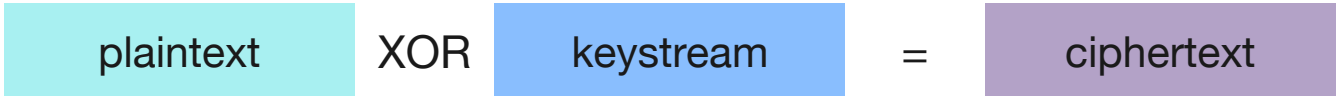
- Binding a key to an identity

  Digital Certificates

Hashes: Detecting Change

Encryption Does Not Prevent Change

A stream cipher combines the message with a keystream, one bit at a time.



Flipping one ciphertext bit flips one plaintext bit



Confidentiality is not integrity

An attacker who cannot read the message can still change it.
If the plaintext is binary data rather than readable text, the recipient may not notice.

You can't tell if encrypted content has been altered

Integrity and Authenticity

Integrity ensures that data has not been modified

Authenticity ensures that data came from the source it claims

- A receiver usually needs **both**:
  - An unchanged message from the expected sender
- **Encryption** provides neither

A software publisher can let anyone download an update	• The content is public • Anyone who downloads it needs to know that it wasn't modified
You're receiving encrypted sensor data via the Internet	• An attacker may intercept & alter it • You need to know it came from the right source and hasn't been modified

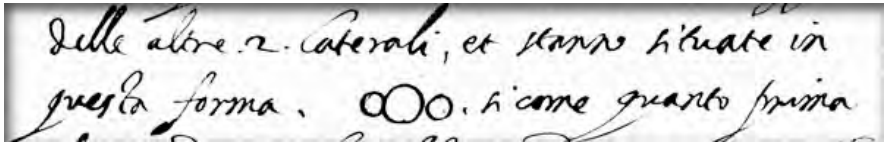
Commitment Scheme

1610: Galileo published to claim a discovery without revealing it:

`smaismrmilmepoetaleumibunenugttauiras`

`Altissimum planetam tergeminum observavi`

“I observed the outermost planet as triple-bodied.”



Hide
The sorted letters reveal nothing about the sentence

Bind
The published value can be revealed as only one message

Robert Hooke's Anagrams

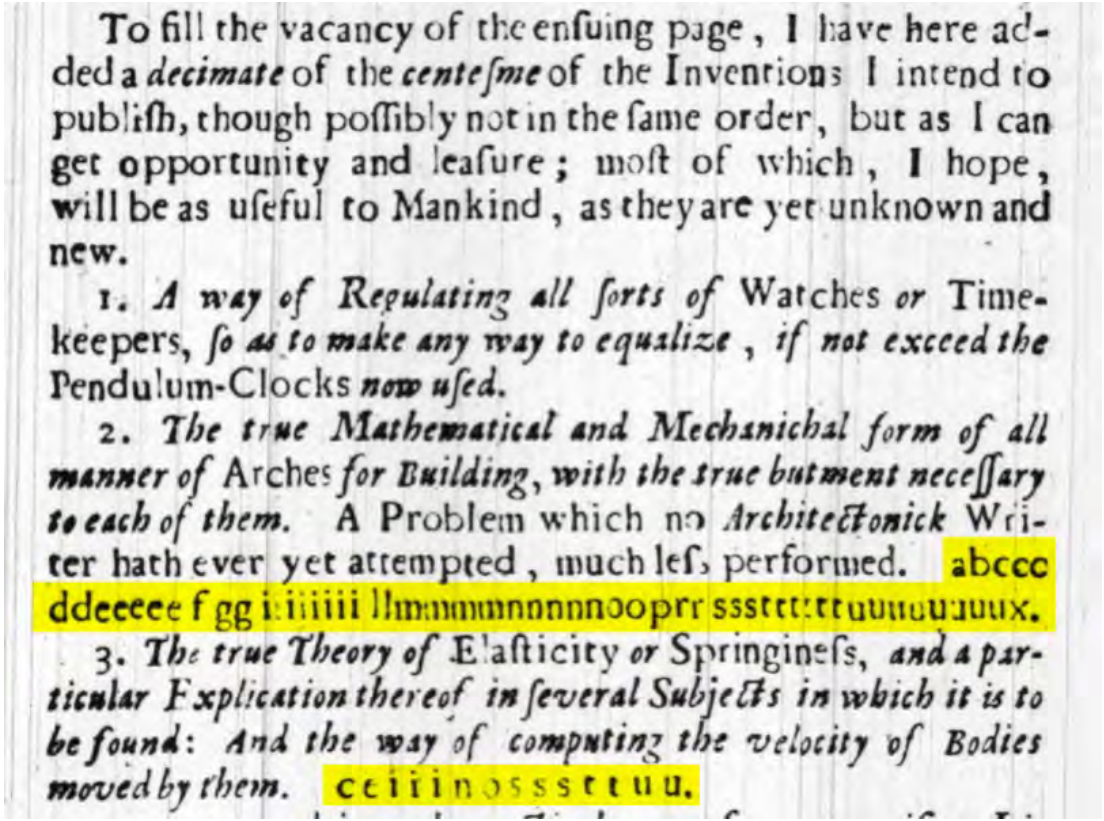
A Description of Helioscopes (1676)

```
ceiiinosssttuu
```

↓

```
Ut tensio, sic vis“
```

“as the extension, so the force.”



To fill the vacancy of the enſuing page, I have here added a *decimate* of the *centeſme* of the Inventions I intend to publiſh, though poſſibly not in the ſame order, but as I can get opportunity and leaſure; moſt of which, I hope, will be as uſeful to Mankind, as they are yet unknown and new.

1. *A way of Regulating all ſorts of* Watches *or* Time-keepers, *ſo as to make any way to equalize, if not exceed the* Pendulum-Clocks *now uſed.*

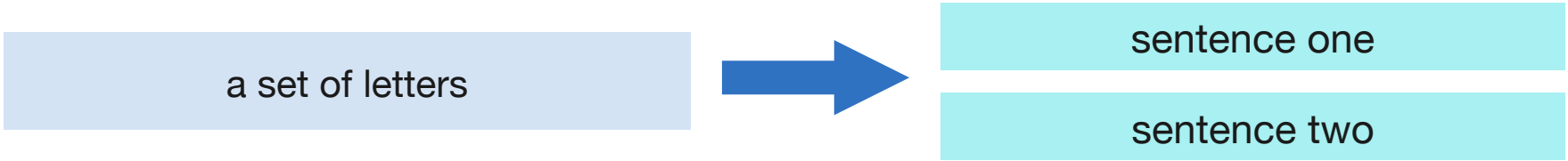
2. *The true Mathematical and Mechanichal form of all manner of* Arches *for Building, with the true butment neceſſary to each of them.* A Problem which no *Architectonick* Writer hath ever yet attempted, much leſs performed. abccc ddeeeee f gg iiiiiii llmmmmnnnnnoopprr ssstttttttuuuuuuuux.

3. *The true Theory of* Elaſticity *or* Springineſs, *and a particular Explication thereof in ſeveral Subjects in which it is to be found: And the way of computing the velocity of Bodies moved by them.* ceiiinosssttuu.

Why Anagram-Based Binding Failed

Binding is where the anagram fails

The published string is a short summary of a longer message.
Binding holds only if nobody can create a second message with the same letters.



Different sentences can often be built from the same letters.

A similar idea appears in every integrity mechanism today

A public procedure reducing a long message to a short summary is a **hash function**.
But we need to fix the binding!

Detecting Accidental Change

- **Commercial telegraphy replaced phrases with five-letter code words**
  - A typo or error in one letter could turn one valid code word into another
  - The good ones separated code words by at least two letters
  - An error in one character could not produce another valid word
  - *Errors in numbers and non-coded words were still a problem*
- **Codebooks created a check word computed from the message**
  - There were many techniques – often kept secret
  - Example: assign numbers to the date and amounts, add a secret value, add numbers together ⇒ look up the number in a codebook to get the check word
- Computing adopted these ideas
  - **Check digits** (credit cards, UPC bar codes), **parity bits**
  - Cyclic redundancy check (**CRC**)

Why Error Detection Is Not Enough

Designed to detect noise errors

A CRC catches accidental corruption. Random corruption is unlikely to produce a value that still passes the check.

Facing an adversary

An attacker changes the data and recomputes the check or crafts a change that leaves the check value unchanged.

The attacker controls both halves of the comparison:

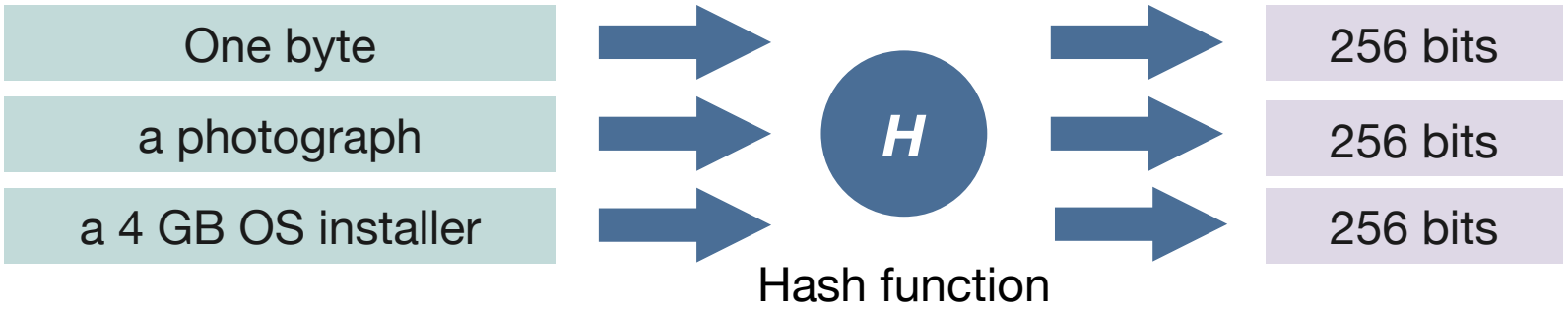


Error detection and tamper detection differ

A check designed for noise assumes the damage is independent of it.

Cryptographic Hash Functions

Any input length maps to one fixed output length:



h = H(m)

Public, and no key, so anyone with the message can compute its **digest**.

SHA-256

Produces 32 bytes, usually written as 64 hexadecimal characters.

Any Change Changes ~50% of the Digest

Two messages differing in one word:

`SEND TEN CRATES`	→	`SEND TWO CRATES`
`5cfa7ec86e3054bc` `9a03bfb5a77de747`	→	`247c7640e6071c0a` `35f9408003d6717c`

122

of 256 digest bits differ

Avalanche

Any change alters about half the digest bits, and which half is unpredictable. Two digests reveal nothing about how similar their inputs were.

Properties of a Cryptographic Hash

Fixed-length	The same size output regardless of input length.
Determinism	The same input always produces the same digest.
Avalanche	Any change to the input changes about half the digest bits.
Efficiency	Fast enough to run over a large file. Work is roughly proportional to the size of the message.

Three more properties describe what an attacker must not be able to do

The Three Resistance Properties

Preimage resistance

Given a digest.

Find any message that produces it.

The digest hides its input.

Protects a message from anyone who sees only its digest

Second preimage resistance

Given one message.

Find another with the same digest.

Blocks file substitution.

Protects a document that exists from outsider modification

Collision resistance

Given nothing.

Find any two messages that match.

The attacker picks both.

Protects against the author creating two versions in advance with the same hash

Why Collisions Must Exist

More possible messages than possible digests

every possible message → 2^{256} digests

Pigeonhole principle

If there are more items than containers, at least one container holds more than one item.
Collisions must exist.



The requirement is not that collisions do not exist

It is that finding a colliding pair takes an infeasible amount of work. Every hash function has collisions. **A secure hash has none that anybody can produce.**

Collisions Are Cheaper Than They Look

Finding a second preimage

Finding a message that produces one chosen digest takes about 2^n attempts for an n-bit digest.

Finding *any* collision

Every new message is compared against every message already tried. About $2^{n/2}$ attempts.

The birthday problem ⇒ hash collisions happen much faster than expected,

Among 23 people, >50% chance that two share the same birthday
The search is for any matching pair, not for a specific date.

But collisions are still infeasible with a good hash function

A digest must be twice as long as the amount of collision resistance wanted.
128 bits give only 64 bits. At 256 bits, 2^{128} work is out of reach.

Popular Hash Functions

Function	Digest size	Status
✖ MD5	128 bits	Collisions found in 2004
✖ SHA-1	160 bits	Collision found in 2017
SHA-2	224, 256, 384, 512 bits	Most widely used
SHA-3	224, 256, 384, 512 bits	In use, different design

MD5 and SHA-1 are no longer considered secure

Neither should be used where collision resistance is required.

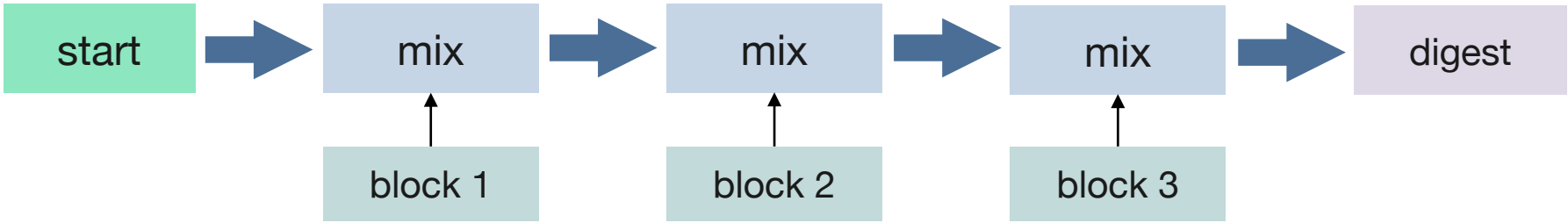
2017 SHA-1 collision

Researchers built two PDF files with the same SHA-1 digest. They chose both files, so this was a collision and not a substitution for an existing file.

- $9x10^{18}$ SHA-1 computations
- 6,500 years of CPU time + 110 years of GPU time

How a Hash Function Is Built

A small function processes the input in blocks, storing intermediate values



One consequence

The digest is the running value at the end of the message, which is everything the function was holding when it finished.

Anyone who has a digest can start the function from that point and keep going, as though more blocks had followed.

What Hash Functions Are Used For

Verifying a transfer

Recompute and compare against a published digest (BitTorrent Linux distributions)

Naming content

Content changes → forces new name. Git and IPFS name data by its digest.

Committing to data

Publish the digest now, reveal the data later. Trusted timestamping services use it.

Storing passwords

Store a digest instead of the password itself.

Linking records

Each block carries the digest of the one before it.

Finding duplicates

Compare digests instead of comparing files.

Two items we'll cover in the next two lectures:

1. Password storage needs a hash , sometimes deliberately made slow
2. Linking records are used with blockchain.

A Digest Needs a Trusted Reference

The publisher's intent:



An attacker can recompute the hash:



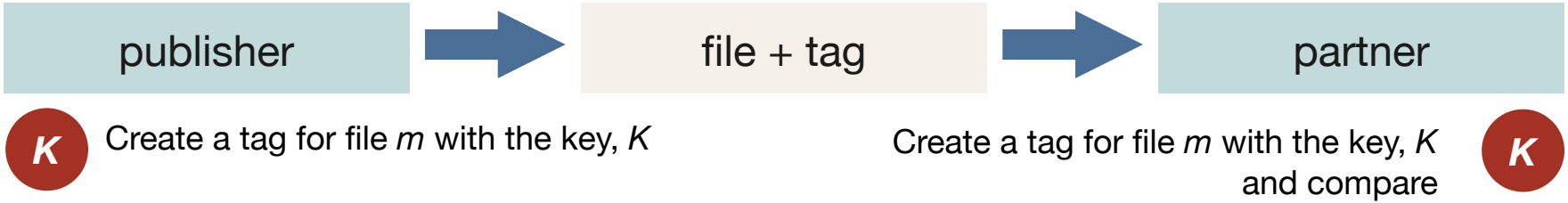
SHA-256 worked correctly both times
The attacker simply recomputed and updated the hash values

What we need:
A check that depends on something the attacker cannot control

Message Authentication Codes: Adding a Secret Key

Message Authentication Codes

Publisher and partner share a randomly generated key:



t = MAC(K, m)

- The sender computes a tag from the message and the key.
- The receiver recomputes it with its own copy of the key and compares.

An outsider does not know the key and cannot compute the tag

A blind guess at an ideal 128-bit tag succeeds with probability 1 in 2^{128}

What a Matching Tag Proves



1. The message was not altered
2. It was produced by someone who knows the key

A hash function on its own provides only the first

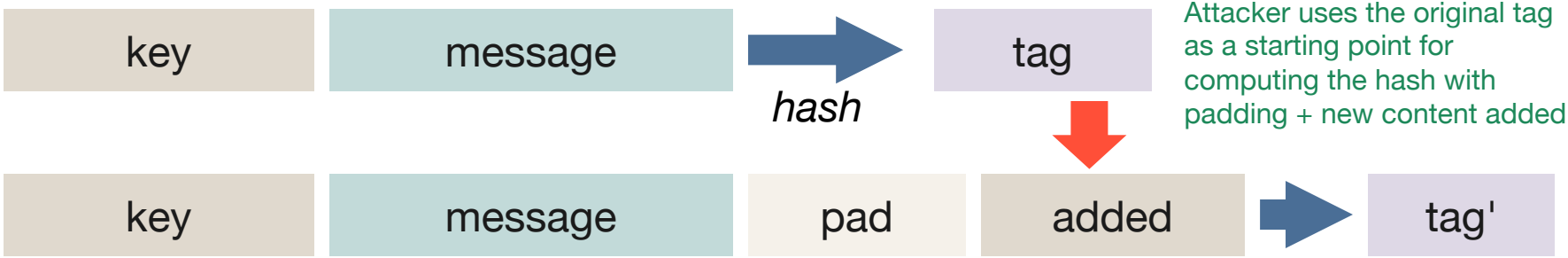
- A MAC provides **integrity** and **authenticity** together
- Banking was the first commercial use

  A transfer instruction between banks is not secret,
  but the amount must arrive as sent

Hashing a Key and Message Is Not a MAC

The obvious construction: hash(key + message)

Put the key in front of the message and hash it. *This has a problem and is not a MAC*



Length extension attack

The tag (hash) is everything the function held when it finished. An attacker resumes from it, appends data, and gets a tag that verifies.

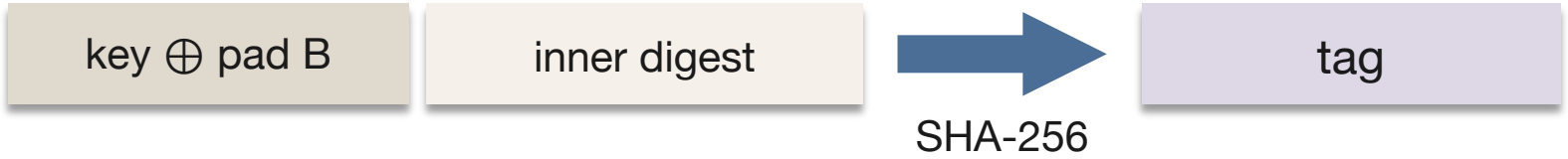
Not a second preimage: a **new** digest for a **new** message.

HMAC (Hash-based Message Authentication Code)

Step 1: Inner hash:



Step 2: Outer hash:



Only the outer digest is the HMAC tag

It is not a state anyone can resume from.
Continuing a hash calculation from the tag will not produce a valid HMAC.

You Need to Protect All The Parts

A tag protects only the bytes used to create it

An update arrives with a file, name, version number, and a tag covering the file data
An attacker changes the name or version number, and the tag still verifies.



Authenticate a **manifest** binding the fields together:

manifest: product name, version, **file digest**	MAC over manifest

Then check the file against the digest

Combining Authentication and Encryption

AEAD: Authenticated Encryption with Associated Data

AEAD produces ciphertext and an authentication tag in one operation

- It can also authenticate readable data, such as a packet header

---

AES-GCM and ChaCha20-Poly1305 do not need a separate HMAC

- **[AES** = block cipher **]** + **[GCM** (Galois Counter Mode) = MAC **]**
- **[ChaCha20** = stream cipher **]** + **[Poly1305** = MAC **]**

TLS 1.3 requires AEAD on every record it sends

AEAD is the default for new designs that send encrypted data

- Older protocols compose a cipher and a MAC separately
- **Encrypt-then-MAC**: tag the ciphertext, so a forgery is rejected before decryption

Shared Secrets Don’t Always Work Well

No non-repudiation

Either key holder could have produced any tag.

A third party cannot tell who did.

Does not scale

If each recipient gets a unique key, we need n keys.

If every pair gets a unique key, we need $n(n\text{-}1)/2$ keys.

Breaks for distribution

A key in every install lets any customer forge updates.

Suppose a publisher ships to 10 million installs

One shared key in every copy lets any customer forge updates.

One key per customer means 10 million secrets to distribute, store, and tag with.

What We Need

A check that anyone can perform and nobody can forge.

Verification must be **public** while **authentication** stays **private**.

Public Keys and Digital Signatures

Public Key Cryptography

One key pair, generated together:



The reverse direction is infeasible: you cannot derive the private key from the public key.

Public key cryptography enables two functions:

Signing

The **private** key produces a signature.

Anyone with the **public** key can verify it.

One signer, any number of verifiers.

Encrypting

Anyone can encrypt with the **public** key.

Only the owner of the **private** key can dercrypt

Anyone can send, one party can read.

The Public-Key Idea

- **By the 1970s networks let people do business remotely**
  - Symmetric cryptography required sharing a secret key in advance
  - Sharing a key is hard when you're not in the same room
    - Network traffic may be sniffed – and you can't encrypt the key
    - Read it over the phone, send it via snail mail
- **In 1976 Diffie and Hellman proposed separating the two operations**
  - They described public-key encryption, digital signatures, and key exchange
  - But they couldn't figure out public key encryption
- **In 1977 Rivest, Shamir, and Adleman produced RSA**
  - Equivalent methods had been found in secret and stayed classified until 1997
  - RSA was patented, which held back adoption – the patent expired in 2000



Exporting code from the U.S. that used the RSA algorithm was a pain because cryptography was on the U.S. Munitions List until 1996.

Export builds of browsers and most other software that used cryptograph were capped at very low-strength values of 512-bit RSA keys and 40-bit symmetric keys

One-Way Functions

Multiplication and factoring

Multiplying two large primes is fast. Recovering them from the product is not.

Modular exponentiation

Raising a number to a power is fast. Recovering the exponent is the discrete logarithm problem.

Elliptic curve multiplication

Adding a point to itself many times is fast. Recovering the count is believed infeasible.

A cryptographic hash

Computing a digest is fast. Preimage resistance is the requirement that reversing it is infeasible.

None of these is *proven* hard

They are believed hard because they have resisted concentrated effort for decades, which is the same standard of evidence that supports every cipher in use.

Example: Discrete Logarithm

$$n = a^b \bmod c$$

$$f(x) = 5^x \bmod 23$$

Going forward, with x = 7:

$$f(x) = 5^7 \bmod 23 = 78{,}215 \bmod 23 = 17$$

Going backward, given $f(x) = 17$, find x such that:

$$5^x \bmod 23 = 17$$

without trying different choices for x?

Trapdoor Functions

A one-way function is not enough to build a public key cryptosystem
The legitimate key holder needs to perform the difficult direction efficiently

$n = p \times q$	→	recovering p and q is infeasible
if n and p known	→	q is easy to compute with one division

Trapdoor

Secret information that makes an infeasible calculation easy.

A hash has no trapdoors

Which is why hashing alone cannot produce a key pair.

Three Functions of Public Key Cryptography

1. **Public-key encryption**: confidential delivery to one recipient
   - Does not identify the sender, since anyone can use the public key
2. **Digital signature**: publicly verifiable authentication of a message
   - Leaves the message readable
   - Can only be created by owner of the private key: provides non-repudiation
3. **Key agreement**: establish a shared secret for symmetric encryption
   - Enable two parties to come create a shared key over an insecure channel

Different algorithms are used – but the same public key concepts

RSA Key Generation

1. Choose two large random primes p and q
2. Compute the modulus $n = p \times q$
3. Compute the totient $\phi = (p-1)(q-1)$
4. Choose a public exponent e sharing no factor with ϕ, usually 65537
5. Compute d, the multiplicative inverse of e mod ϕ

Public key is **(e, n)**. **Private key** is **(d, n)**.
Destroy p and q

An example with small numbers.
This isn't secure but the math works.

$p = 61$ and $q = 53$

$n = 3233$

$\phi = 3120$

$e = 65537$

$d = 2753$

Pub key = (65537, 3233)

Pri key = (2753, 3233)

RSA Encryption and Decryption

Encryption and decryption are modular exponentiation operations

Encrypt: $\text{plaintext}^{\text{public key}} = C = P^e \bmod n$

Decrypt: $\text{ciphertext}^{\text{private key}} = P = C^d \bmod n$

Using our example: *public key* $(e, n) = (65537, 3233)$ *private key* $(d, n) = (2753, 3233)$

Suppose P=123

Encrypt:	P = 123	→	$123^{65537} \bmod 3233$	→ C=855
Decrypt:	C = 855	→	$855^{2753} \bmod 3233$	→ C=123

A real modulus is at least 2048 bits

Factoring is much easier than trying every key, so the modulus must be far larger than a symmetric key for the same security.

RSA Security

Since the security of RSA rests on not knowing an efficient way to factor $n = pq$, we have to make it difficult to find factors via an exhaustive search

Common key lengths are 2048, 3072, 4096, 8129, 16384 bits

Example: a 2048-bit modulus (n) and secret exponent (d):

$n =$
0xa709e2f84ac0e21eb0caa018cf7f697f774e96f8115fc2359e9cf60b1dd8d4048d974cdf8422bef6be3c162b04b916f7ea213
3f0e3e4e0eee164859bd9c1e0ef0357c142f4f633b4add4aab86c8f8895cd33fbf4e024d9a3ad6be6267570b4a72d2c34354e0
139e74ada665a16a2611490debb8e131a6cffc7ef25e74240803dd71a4fcd953c988111b0aa9bbc4c57024fc5e8c4462ad904
9c7f1abed859c63455fa6d58b5cc34a3d3206ff74b9e96c336dbacf0cdd18ed0c66796ce00ab07f36b24cbe3342523fd8215a8
e77f89e86a08db911f237459388dee642dae7cb2644a03e71ed5c6fa5077cf4090fafa556048b536b879a88f628698f0c7b420
c4b7

$d =$
0x10f22727e552e2c86ba06d7ed6de28326eef76d0128327cd64c5566368fdc1a9f740ad8dd221419a5550fc8c14b33fa9f058
b9fa4044775aaf5c66a999a7da4d4fdb8141c25ee5294ea6a54331d045f25c9a5f7f47960acbae20fa27ab5669c80eaf235a1d0
b1c22b8d750a191c0f0c9b3561aaa4934847101343920d84f24334d3af05fede0e355911c7db8b8de3bf435907c855c3d7eeee
de4f148df830b43dd360b43692239ac10e566f138fb4b30fb1af0603cfcf0cd8adf4349a0d0b93bf89804e7c2e24ca7615e51af
66dccfdb71a1204e2107abbee4259f2cac917fafe3b029baf13c4dde7923c47ee3fec248390203a384b9eb773c154540c5196
bce1

The size of each value is 10^{617}

Weak RSA Public Keys

March 14, 2022

- Older software generated RSA keys that can be broken instantly with commodity hardware
- SafeZone library doesn't randomize the prime numbers well
  - Used to generate RSA keys
  - After selecting one prime #, the second one is in close proximity to the first
- Keys generated with primes that are too close together can be broken with Fermat's factorization method, described in 1643



ars TECHNICA SUBSCRIBE SIGN IN

BREAKING KEYS —

Researcher uses 379-year-old algorithm to crack crypto keys found in the wild

It takes only a second to crack the handful of weak keys. Are there more out there?

DAN GOODIN - 3/14/2022, 5:31 PM

Getty Images

Enlarge

Cryptographic keys generated with older software now owned by technology company Rambus are weak enough to be broken instantly using commodity hardware, a researcher reported on Monday. This revelation is part of an investigation that also uncovered a handful of weak keys in the wild.

The software comes from a basic version of the SafeZone Crypto Libraries, which were developed by a company called Inside Secure and acquired by Rambus as part of its 2019 acquisition of Verimatrix, a Rambus representative said. That version was deprecated prior to the acquisition and is distinct from a FIPS-certified version that the company now sells under the Rambus FIPS Security Toolkit brand.

Elliptic Curve Cryptography

Elliptic-curve cryptography

The public value is calculated from a secret number using arithmetic on the points of an elliptic curve. Recovering the secret number is believed infeasible.

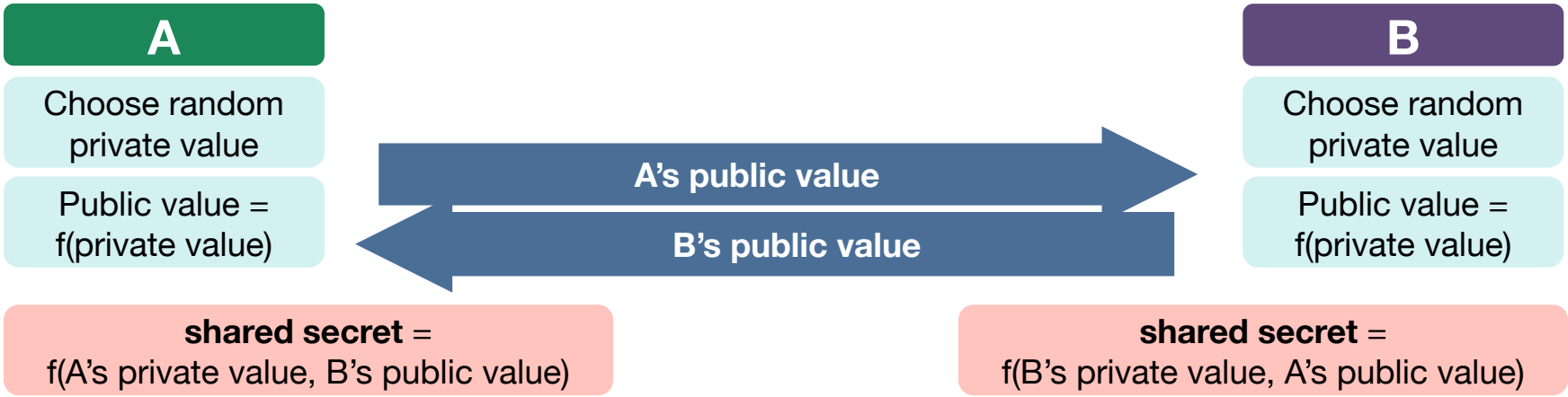
Comparable security, much smaller keys:

security level	RSA modulus	elliptic curve
112 bits	2048 bits	224 bits
128 bits	3072 bits	256 bits

Smaller keys mean less data on the wire and faster operations

Diffie-Hellman Key Exchange

Each party picks a private value, computes and sends a public value



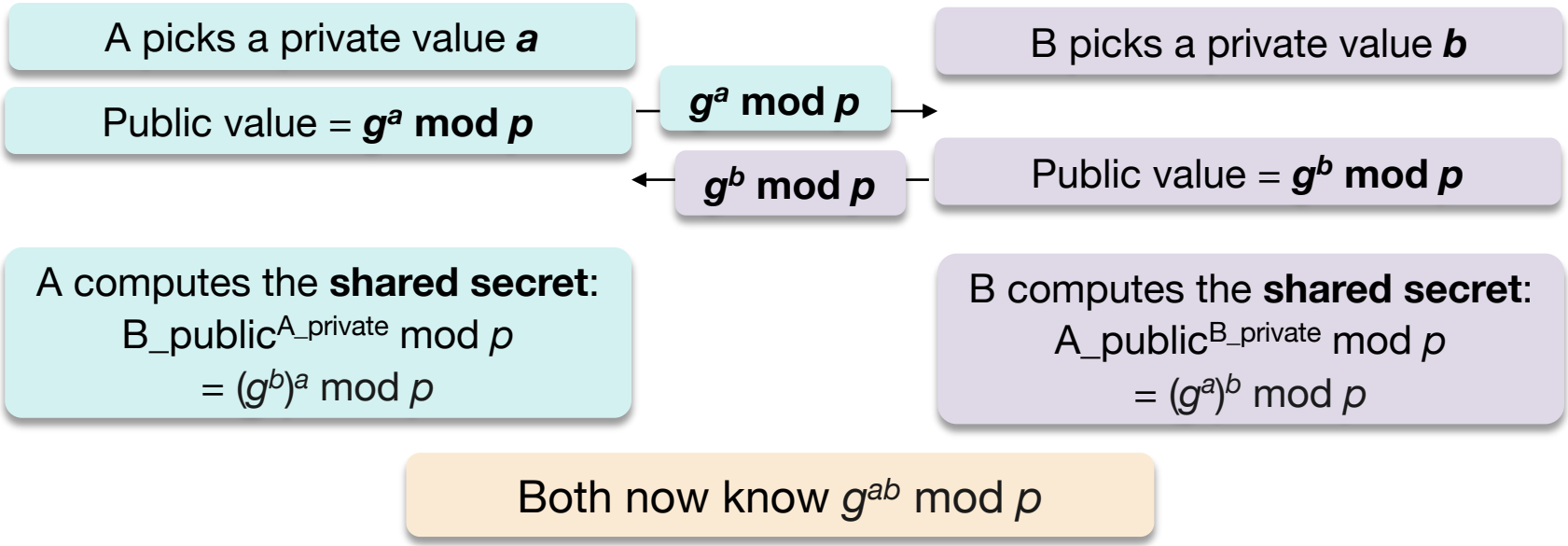
A & B derive the same shared secret

The shared secret is never transmitted
Neither party sends the result.

Nobody is authenticated
It does not say who is at the other end.

The Diffie-Hellman Calculation

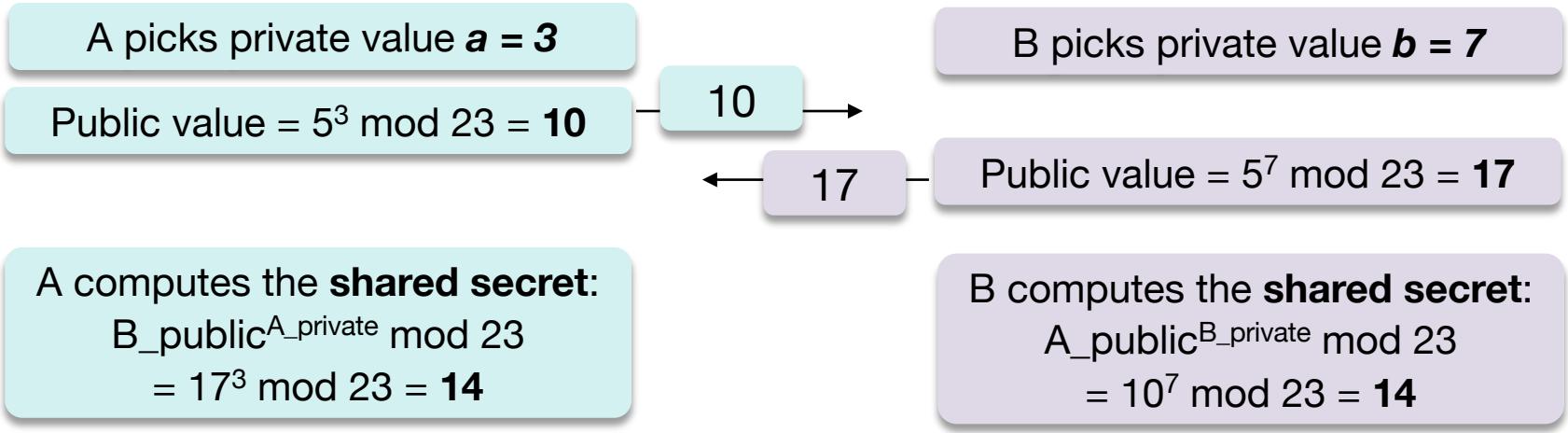
Agree on a large prime ***p*** and a base **g** in advance. Neither is secret.



Recovering a from $g^a \bmod p$ is the discrete logarithm problem
Elliptic-curve Diffie-Hellman does the same exchange with curve arithmetic.

Diffie-Hellman Example (Tiny numbers)

Agree on a large prime ***p* = 23** and a base **g = 5**. Neither is secret.



An attacker cannot compute this by seeing only the public values

Why Key Exchange Uses Diffie-Hellman

Per operation RSA encryption and Diffie-Hellman are comparable
Both are modular exponentiation on similar-sized numbers at equal security.

A fresh RSA key pair
Search for two large primes.

Too slow to do per connection.

A fresh Diffie-Hellman key
One random number, one exponentiation.

Cheap on every connection.

Fresh key material per session becomes affordable (low-latency)

Diffie-Hellman generates a new key pair cheaply enough to do it on every connection. Elliptic curves are even faster, with much smaller public values.

Why Not Encrypt Everything With Public Key Crypto?

- **Speed**: AES runs in hardware at gigabytes per second
  - Elliptic curves beat RSA and are still thousands of times slower than AES
- **Overhead**: RSA ciphertext is one full modulus per block, 256 bytes at 2048 bits
  - Elliptic curve encryption adds a fixed overhead, not a multiple
- **No randomization**: the encryption key is public, so guesses can be tested
- **Structure survives**: relationships in the plaintext appear in the ciphertext
  - Multiplying two raw RSA ciphertexts encrypts the product of their plaintexts

Public key algorithms were designed to distribute keys and sign values, not to encrypt bulk data

Digital Signatures

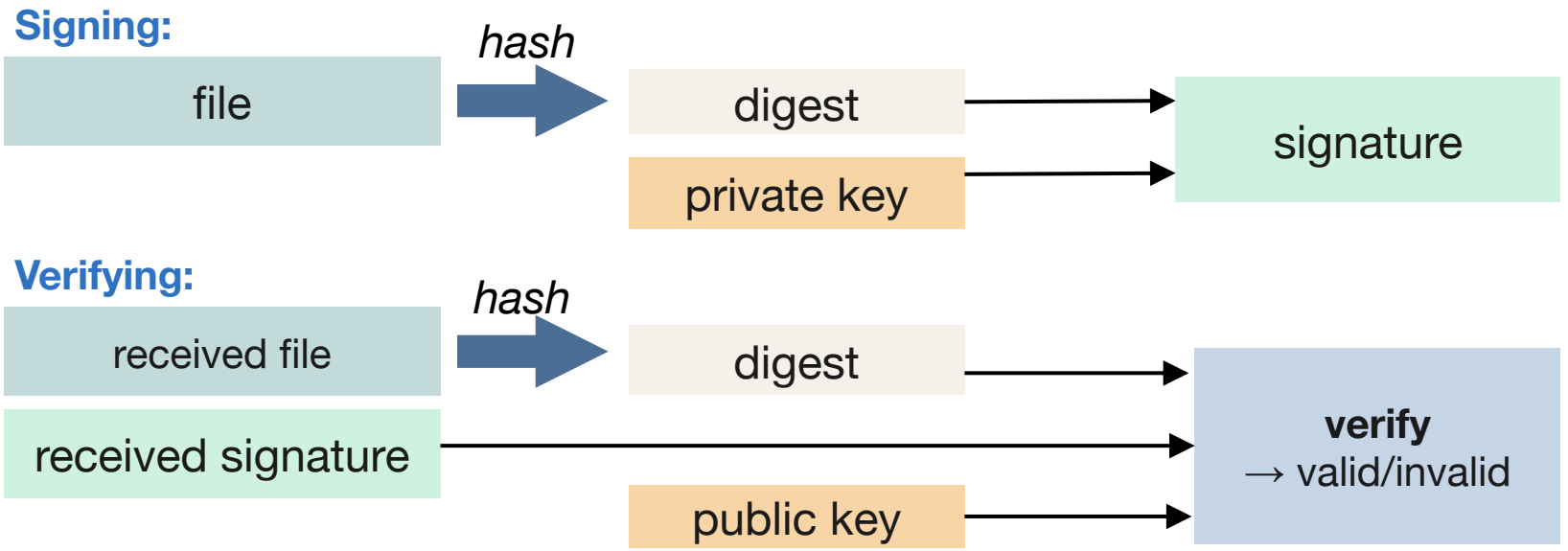
MACs did not prove *who* created the tag & you need a shared key to validate

- **Digital signatures** ensure that there's one signer and anyone can validate

- A value computed from a message and a private **signing key**
- Anyone holding the matching **verification key** can check it
- Three operations define the scheme
  - **Key generation**: produce a private key and its related public key
  - **Signing**: private key plus message produces a fixed-length signature
  - **Verification**: public key, message, and signature give a yes or no

A valid result means the message is unaltered and came from the key holder

Signing and Verification



A mirror cannot alter the release undetectably

Substituting a modified file needs a valid signature for the new contents, and producing one without the private key is infeasible.

Hashing the Message

The signature is computed over the digest

An update may be gigabytes. The public-key step works on a short value.

Why collision resistance is important, and not only second preimage resistance:

1. The attacker writes two documents that share a digest: one harmless, one fraudulent
2. The signer reviews the harmless document and signs its digest
3. The same signature verifies on the fraudulent document, which the signer never saw

The attacker chose both documents

Matching a document that already exists takes about 2^n attempts. Writing both cuts it to about $2^{n/2}$. A weakness in the function itself cuts it further. That is how MD5 and SHA-1 fell.

Signature Schemes in Use

- For RSA only, signing resembles encrypting a hash with the private key
  - The same modular exponentiation, with the two exponents swapped
- The analogy does not generalize
  - RSA signing uses different padding than RSA encryption
  - ECDSA and Ed25519 have no encryption operation at all
  - In general:
    verification checks for a relationship between {*message*, *signature*, *public key*}

It's a good mental model but no longer accurate in many implementations

- **RSA-PSS** combines hashing with a randomized encoding
- **ECDSA** appears in TLS certificates, in Bitcoin, and in code signing
- **Ed25519** is deterministic, removing a class of implementation failure

Signatures From Hash Functions Alone

A signature scheme needs no number theory
In 1979 Lamport described one built from nothing but a one-way function.

To sign one bit:

generate two random values → *hash* → publish both digests as public key

reveal one value to sign the bit

verifier hashes it and compares

Impractical alone, but the foundation of something current

Keys are large and each pair signs one message.
Combining many pairs of keys into a tree fixes that, and the result became one of the post-quantum signature schemes.

You don't have to understand this!!

What a Signature Gives Us

Non-repudiation

Only the key holder can produce it, anyone can check it, and the recipient cannot forge it. *The signer cannot credibly deny it.*

A signature does not establish truth or safety

A valid signature does not mean the message is true or the software is safe. A publisher can sign a program with a serious vulnerability

A signature does identify the signer

A valid signature says only that signature and key match. Anyone can make a key pair.

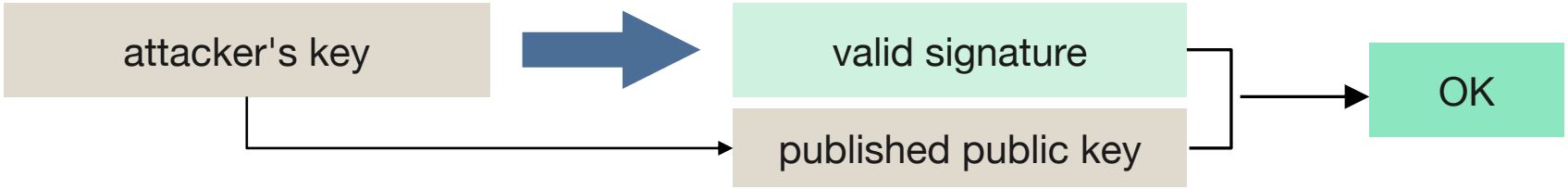
Whose key is it?

Certificates:
Trusting Keys and Signed Software

A Valid Signature From the Wrong Key

A valid signature can come from the wrong person

An attacker replaces the software, signs it with a new key, and *publishes that key as the publisher's*.



We need to establish that the public key belongs to the real publisher

A trusted channel

A publisher can ship its verification key for future updates inside the installer.

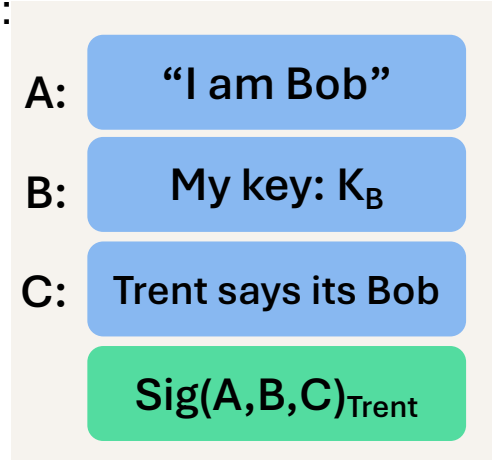
Compare a key fingerprint

A hash of the public key, short enough to compare by hand.

Browsers need something that scales!

Identity Binding – Another Option

- Have a trusted party (Trent) sign Bob's public key
- Once signed, it is tamper-proof
  - An attacker cannot generate the signature after modifying the key
- But we need to know it's Bob's public key and who signed it
  - Create & sign a data structure that:
    A. Identifies Bob
    B. Contains his public key
    C. Identifies who is doing the signing



Everyone trusts Trent

Digital Certificates

- A **certificate authority** (**CA**) checks a binding
  - It signs a statement vouching for the connection between a **key** and a **name**
  - It's a trusted third party
- That signed statement is a **digital certificate**
  - **X.509** is the format used on the web
- Some of the data it contains:

subject	public key	Issuer (CA)	CA signature
serial #	validity period	extensions	signature algorithm

Certificate Chains

Each authority vouches for the key below it:

Trust anchor: trusted root key (stored in OS or browser)

verifies the intermediate authority's certificate

whose key verifies the website's certificate

A root is trusted because it is in the trust store

Not because it is self-signed. Anyone can self-sign.

Example Certificate Chain

The server (www.google.com) sends the first two certificates.
The root certificate is stored in the operating system's or browser's **trust store**

www.google.com certificate

subject: `www.google.com`	Issuer: WR2 (Google Trust Services)	WR2's signature

Intermediate certificate

subject: `WR2`	Issuer: GTS Root R1 (Google Trust Services)	Trusted root CA

Trust Store

subject: GTS Root R1	Issuer: GTS Root R1 (self-signed)	GTS Root R1

Verifying a Certificate

- Check the validity period, and that the name appears in the certificate
- Verify each signature with the public key in the certificate of the CA that issued this certificate
- Check that each intermediate is authorized to issue certificates (a *Basic Constraints* extension marks it as a CA)
- Confirm the chain ends at a root in the **trust store**
- Check revocation status
- Confirm the other party holds the **private key**, by signing fresh data
  - The other side should prove that it can sign something it hasn't seen before

Revocation and Transparency

Certificate Revocation Lists (CRLs)
A signed list of withdrawn serial numbers. Lists grow large and go stale.

Online status checks
A live query about one certificate. It leaks browsing behavior, adds a round trip, and is treated as passing when it fails.

The practical response was to work around the problem
The server can send a recent signed status with the connection, and certificate lifetimes are now months rather than years, which narrows the window a stolen key is useful.

Certificate Transparency

- Any authority in the trust store can issue a certificate for any name.
- Public append-only logs record every certificate issued, so a usable one cannot be **hidden** from the name's owner.

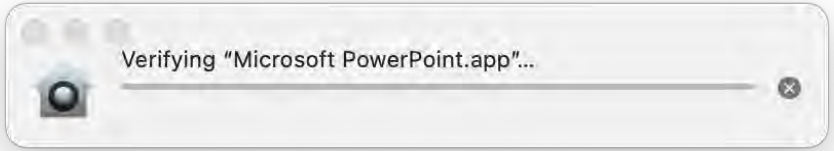
Code Signing

At build time:

binary → digest → signature + chain

At install or load time:

recompute digest | verify signature | verify chain to root



Per-page hashes
Pages are checked as they load.

Secure boot
Firmware verifies the bootloader, which verifies the kernel.

What a Valid Signature Does Not Prove

- **It does not say the software is safe**
  - A publisher can sign a program with a serious vulnerability
- **It does not say the publisher knew what they were signing**
  - A build system hands a binary to a signing key
  - Anything inserted before that point is signed without anyone noticing

The result is correctly signed malicious code that passes every check

Three Ways Into a Signed Release

The signing key
A key is stolen, or a weak hash forges a certificate.

Cryptography failed.

The build system
Code is inserted during compilation, and the vendor signs the result.

Cryptography was perfect.

A dependency
An attacker becomes a maintainer and publishes legitimately.

Nothing was tested.

Signing proves the last link of the chain
The rest of the chain became the target.

Defenses attack different parts
Reproducible builds, provenance records, transparency logs, and protected keys.

Quantum Computers & Cryptography

Once (if) useful quantum computers can be built, they will be able to:

- Factor efficiently
  - Shor's algorithm on quantum computers factors numbers exponentially faster
  - RSA will not be secure anymore
- Find discrete logarithms & elliptic curve discrete logarithms efficiently
  - Diffie-Hellman key exchange & ECC will not be secure



Affects the trapdoor functions that public key cryptography relies on

Not all is bad

Symmetric cryptography is largely immune to attacks

Some optimizations are predicted

Grover's algorithm: square-root speedup

Crack a symmetric cipher in time proportional to the square root of the key space size: $2^{n/2}$ vs. 2^{n}

The recommendation is to use 256-bit AES to be safe



The State of Post-Quantum Cryptography

Symmetric ciphers and hashes

No replacement is needed. Larger parameters are the whole response, and most systems already use them.

Public key algorithms

Replacement needed. New standards cover key establishment and signatures.

Harvest now, decrypt later

Recording traffic is cheap, so confidentiality must migrate before the hardware exists. Attackers can capture traffic and decrypt years later. Digital signatures are under less pressure.

Cautious approach: hybrid deployment

Derive a key from bot a classical exchange and a post-quantum one.
The connection will stay secure unless both are broken.

One new signature scheme is based only on hash functions.

NIST Releases First Post-Quantum Encryption Standards

August 13, 2024: First set of post-quantum standards released:
CRYSTALS-Kyber, CRYSTALS-Dilithium, Sphincs+ and FALCON

1. FIPS 203 - ML-KEM – **key establishment**
   Module-Lattice-Based Key-Encapsulation Mechanism,
   based on CRYSTALS-Kyber algorithm

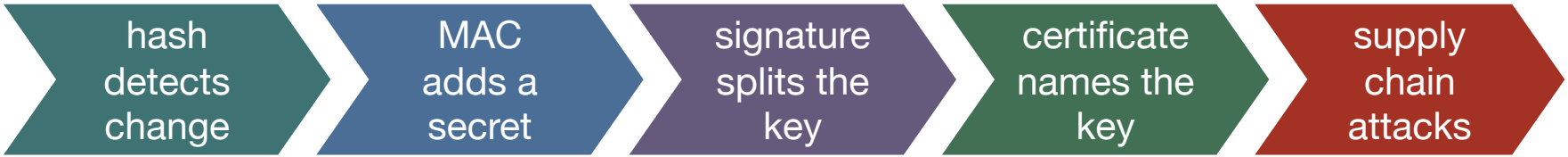
2. FIPS 204 – ML-DS - **signatures**
   Module-Lattice-Based Digital Signature Algorithm,
   based on the CRYSTALS-Dilithium algorithm

3. FIPS 205 – SLH-DSA – **signatures –** *backup to ML-DS*
   Stateless Hash-Based Digital Signature Algorithm)

4. (draft) FIPS 206 – **signatures –** FN-DSA
   Fast-Fourier Transform over NTRU-Lattice-Based Digital Signature Algorithm

5. (2025 proposal) HQC – **key encapsulation –** *backup to ML-KEM*
   Key Encapsulation Mechanism based on error correcting codes

Where We Ended Up



Each step exists because the one before it ran out

A hash cannot say ***who***. A shared secret cannot serve millions. A signature cannot say whose key. A certificate cannot say what the build contained.

The conservative post-quantum scheme

It uses no number theory and is is built from a hash function in a tree of one-time keys.

The first tool introduced needed replacing **least**.

The End